

Day 4: Dictionaries & Exception Handling

Welcome to Day 4! Today we cover two essential pillars of robust Python programming:

1. **Dictionaries:** Python's native implementation of associative arrays or hash maps.
 2. **Exception Handling:** The mechanism to handle runtime errors gracefully, keeping programs running under unexpected conditions.
-

Part 1: Associative Arrays (Dictionaries)

1. Introduction to Dictionaries

A **dictionary** in Python is an unordered collection (insertion-ordered starting from Python 3.7) of items. Each item is stored as a **key-value pair**.

- **Key:** Must be unique and **hashable** (immutable types such as strings, numbers, or tuples containing only immutable elements).
- **Value:** Can be of any arbitrary Python data type (lists, dictionaries, integers, custom objects, etc.) and does not need to be unique.

Dictionaries are optimized for retrieving data. Under the hood, Python uses a hash table structure, allowing lookup, insertion, and deletion operations in average $O(1)$ time complexity.

2. Defining Dictionaries

There are multiple ways to define a dictionary:

```
# 1. Empty dictionary
empty_dict_1 = {}
empty_dict_2 = dict()

# 2. Dictionary literal with data
student = {
    "name": "Arham",
    "age": 21,
    "course": "PGCP-AI",
    "grades": [85, 90, 88]
}

# 3. Using the dict() constructor with keyword arguments
employee = dict(name="Lisa", id=1042, department="R&D")

# 4. Using dict() with list of tuples (key-value pairs)
colors = dict([("red", "#FF0000"), ("green", "#00FF00"), ("blue", "#0000FF")])

print("Student:", student)
```

```
print("Employee:", employee)
print("Colors:", colors)
```

3. Accessing Items

You can access values using their corresponding keys. Python offers two primary methods:

A. Bracket Notation (`dict[key]`)

Directly look up a key. If the key does not exist, Python raises a `KeyError`.

```
profile = {"username": "vinod_k", "role": "admin"}

# Valid access
print(profile["username"]) # Output: vinod_k

# Invalid access (raises KeyError)
try:
    print(profile["email"])
except KeyError as e:
    print(f"KeyError caught: Key {e} does not exist.")
```

B. The Safe `.get()` Method

Returns the value if the key exists; otherwise, returns `None` or a specified default value. It **never** raises a `KeyError`.

```
profile = {"username": "vinod_k", "role": "admin"}

# Safe retrieval
email = profile.get("email")
print("Email:", email) # Output: Email: None

# Safe retrieval with custom default
email_with_default = profile.get("email", "no-email@example.com")
print("Email (with default):", email_with_default) # Output: no-email@example.com
```

C. Retrieving Views (`.keys()`, `.values()`, and `.items()`)

These methods return dynamic view objects that reflect dictionary changes in real time.

```
inventory = {"apples": 10, "bananas": 24}

# View of keys
keys_view = inventory.keys()
```

```
print("Keys:", list(keys_view)) # Output: ['apples', 'bananas']

# View of values
values_view = inventory.values()
print("Values:", list(values_view)) # Output: [10, 24]

# View of key-value tuples
items_view = inventory.items()
print("Items:", list(items_view)) # Output: [('apples', 10), ('bananas', 24)]
```

4. Modifying and Adding Items

Dictionaries are mutable. You can add new key-value pairs or update existing ones.

```
car = {"brand": "Tesla", "model": "Model 3"}

# Adding a new key-value pair
car["year"] = 2023

# Modifying an existing value
car["model"] = "Model S"

# Using the .update() method to add/modify multiple items at once
car.update({"color": "red", "year": 2024})

print("Updated Car:", car)
# Output: {'brand': 'Tesla', 'model': 'Model S', 'year': 2024, 'color': 'red'}
```

5. Deleting Items

Python provides several ways to delete entries:

```
stats = {"HP": 100, "MP": 50, "Speed": 75, "Defense": 60}

# 1. del keyword: Removes key-value pair. Raises KeyError if key doesn't exist.
del stats["Defense"]

# 2. .pop(): Removes key and returns its value. Returns default if key is not found (avoids KeyError).
mp_value = stats.pop("MP")
print(f"Popped MP value: {mp_value}")

speed_fallback = stats.pop("Stamina", 0) # Stamina is not in stats, returns 0
print(f"Popped Stamina (fallback): {speed_fallback}")
```

```
# 3. .popitem(): Removes and returns the last inserted key-value pair as a
tuple.
last_item = stats.popitem()
print(f"Popped last item: {last_item}") # Output: ('Speed', 75)

# 4. .clear(): Wipes the entire dictionary, making it empty.
stats.clear()
print("Cleared stats:", stats) # Output: {}
```

6. Dictionary Comprehensions

Similar to list comprehensions, dictionary comprehensions provide a concise way to construct dictionaries from iterables.

Syntax:

```
{key_expression: value_expression for item in iterable if condition}
```

Example:

```
# Create a dictionary of squares for even numbers from 1 to 10
squares = {x: x**2 for x in range(1, 11) if x % 2 == 0}
print("Even Squares:", squares)
# Output: {2: 4, 4: 16, 6: 36, 8: 64, 10: 100}

# Inverting a dictionary (assuming unique values)
original = {"a": 1, "b": 2, "c": 3}
inverted = {value: key for key, value in original.items()}
print("Inverted:", inverted)
# Output: {1: 'a', 2: 'b', 3: 'c'}
```

7. Iterating Through Dictionaries

You can loop through a dictionary in different ways:

```
user_roles = {"alice": "manager", "bob": "developer", "charlie": "tester"}

print("--- Iterating over Keys (Default) ---")
for name in user_roles:
    print(name)

print("\n--- Iterating over Values ---")
for role in user_roles.values():
    print(role)

print("\n--- Iterating over Key-Value Pairs ---")
```

```
for name, role in user_roles.items():  
    print(f"User: {name} | Role: {role}")
```

Part 2: Exception Handling

1. Understanding Exceptions

An **exception** is an error that occurs during the execution of a program (runtime). When Python encounters an error it cannot handle, it creates (or "raises") an exception object. If unhandled, the program terminates abruptly (crashes).

Common built-in exceptions include:

- **ZeroDivisionError**: Raised when dividing a number by zero.
- **ValueError**: Raised when a function receives an argument of correct type but inappropriate value (e.g., trying to convert "abc" to an integer).
- **KeyError**: Raised when a dictionary key is not found.
- **IndexError**: Raised when a sequence subscript is out of range.
- **TypeError**: Raised when an operation is applied to an object of inappropriate type.
- **FileNotFoundError**: Raised when a file or directory is requested but does not exist.

2. The **try-except** Block

To prevent crashes, wrap error-prone code inside a **try** block, and handle potential errors inside one or more **except** blocks.

```
try:  
    number = int(input("Enter an integer: "))  
    result = 100 / number  
    print(f"Result: {result}")  
except ValueError:  
    print("Error: That was not a valid integer!")  
except ZeroDivisionError:  
    print("Error: Cannot divide by zero!")
```

Catching Multiple Exceptions in a Single Block

You can group multiple exceptions into a tuple if they share the same handling logic:

```
try:  
    # Potentially problematic operations  
    data = [10, 20]  
    val = data[5] / 0  
except (IndexError, ZeroDivisionError) as e:  
    print(f"An index or arithmetic error occurred: {e}")
```

Catching All Exceptions (Generic Catch)

Use a generic `except Exception as e` to catch all standard errors. Avoid using a bare `except:` as it catches system-exiting signals (`SystemExit`, `KeyboardInterrupt`), which makes stopping your program with `Ctrl+C` difficult.

```
try:
    x = 1 / 0
except Exception as e:
    print(f"Something went wrong: {e}")
```

3. The `else` Clause

The `else` block runs **only if no exceptions were raised** in the `try` block. It is useful for separating the code that might cause exceptions from code that should execute only upon successful completion.

```
try:
    file_content = "105"
    number = int(file_content)
except ValueError:
    print("Could not parse file content as an integer.")
else:
    # Executes only if no ValueError occurred
    print(f"Successfully parsed number: {number}")
    double_val = number * 2
    print(f"Double: {double_val}")
```

4. The `finally` Clause (Cleanup)

The `finally` block **always executes**, regardless of whether an exception was raised, caught, or completely unhandled. It is primarily used to release external resources (like files, database connections, or network sockets).

```
try:
    print("Opening transaction log...")
    # Simulate a crash inside the try block
    result = 1 / 0
except ZeroDivisionError:
    print("Handling division by zero...")
finally:
    # This block executes no matter what
    print("Closing transaction log safely. Done.")
```

Output:

```
Opening transaction log...
Handling division by zero...
Closing transaction log safely. Done.
```

5. Raising Exceptions (`raise`)

You can manually trigger an exception using the `raise` keyword. This is useful for enforcing business rules or validating function arguments.

```
def set_percentage(value):
    if value < 0 or value > 100:
        raise ValueError("Percentage must be between 0 and 100 inclusive.")
    print(f"Percentage set to: {value}%")

try:
    set_percentage(150)
except ValueError as e:
    print(f"Validation failed: {e}")
```

6. Custom Exceptions

You can define custom exceptions to represent errors specific to your application domain. To do this, inherit from the built-in `Exception` class.

```
# Define a custom exception class
class InsufficientFundsError(Exception):
    """Raised when an account withdrawal exceeds the available balance."""
    def __init__(self, balance, amount):
        super().__init__(f"Attempted to withdraw ${amount} with a balance of ${balance}.")
        self.balance = balance
        self.amount = amount

# Usage
def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientFundsError(balance, amount)
    return balance - amount

try:
    current_balance = 50
    new_balance = withdraw(current_balance, 75)
except InsufficientFundsError as e:
    print(f"Transaction Rejected: {e}")
```

7. Behavior of `return` in `try-except-finally`

A common conceptual pitfall: **What happens if a function executes `return` statements inside both the `try` (or `except`) block AND the `finally` block?**

Rule: The `finally` block's `return` statement will override any prior `return` statements or active exceptions in the `try` or `except` blocks.

```
def check_return_behavior():
    try:
        print("Inside try block")
        return "Return from try"
    except Exception:
        return "Return from except"
    finally:
        print("Inside finally block")
        return "Return from finally" # This overrides the try block's
return

result = check_return_behavior()
print("Result of function call:", result)
```

Output:

```
Inside try block
Inside finally block
Result of function call: Return from finally
```

[!WARNING] Putting `return` statements inside `finally` blocks is generally discouraged because it can suppress unhandled exceptions silently, making debugging difficult.

Practical Examples (Interactive & Runnable)

Example 1: Document Word Frequency Counter

A complete program that processes text to count words, utilizing string methods, dictionary operations, and sorting.

```
def count_word_frequencies(paragraph):
    # Dictionary to hold the word counts
    word_counts = {}

    # Preprocessing: remove punctuation, convert to lowercase, and split
    cleaned_text = ""
    for char in paragraph.lower():
        if char.isalnum() or char.isspace():
            cleaned_text += char
        else:
            cleaned_text += " " # Replace punctuation with spaces
```

```
words = cleaned_text.split()

# Counting frequencies
for word in words:
    # Using get() to safely handle initial counting
    word_counts[word] = word_counts.get(word, 0) + 1

return word_counts

# Run Example
sample_text = "Python is amazing! Python is fast, and Python is easy to learn."
frequencies = count_word_frequencies(sample_text)

# Sort dictionary by value (frequencies) in descending order
sorted_frequencies = dict(sorted(frequencies.items(), key=lambda item: item[1], reverse=True))

print("Word Frequencies:")
for word, count in sorted_frequencies.items():
    print(f" - {word}: {count}")
```

Example 2: Robust Numeric Input Reader

An interactive loop that guarantees retrieval of a valid number from user terminal input.

```
def read_valid_integer(prompt, min_val=0, max_val=100):
    while True:
        try:
            user_input = input(prompt)
            # Try to convert input to integer
            value = int(user_input)

            # Business rule validation
            if value < min_val or value > max_val:
                raise ValueError(f"Value must be between {min_val} and {max_val} inclusive.")

        except ValueError as err:
            # Catches both non-numeric text and values outside range
            print(f"Invalid input: {err}. Please try again.\n")
        else:
            # Executes only if conversion and validation succeeded
            print("Input successfully accepted!")
            return value
```